

Stacking Ensemble Approach for AI Generated Text Detection with Appropriate Feature Selection

Md. Siam Ansary

Institute of Information Technology (IIT), University of Dhaka (DU)

Department of CSE, Ahsanullah University of Science and Technology (AUST)

Dhaka, Bangladesh

ORCID: 0000-0002-4942-2541

Abstract—With the growing prominence of artificial intelligence, its integration into everyday life has become increasingly widespread, often in ways once thought unimaginable. Today, AI tools are routinely employed to generate diverse forms of text, such as academic articles and business reports. While these systems demonstrate impressive capabilities, the ability to distinguish between human-written and AI-generated content has become crucial to prevent misuse and deception. In this study, we propose a Stacking Ensemble Approach that leverages seven strong base learners and uses a meta-model. We utilized publicly available datasets from Kaggle and conducted extensive feature engineering to extract key indicators relevant to the detection task. Our methodology has achieved near-perfect performance across standard evaluation metrics. To support transparency and reproducibility, the code used in our experiments will be made publicly accessible.

Index Terms—agt detection, nlp, text classification

I. INTRODUCTION

Artificial intelligence has experienced significant advancement in the past few decades in a range of fields. The use of AI is currently being applied in the manner that did not exist before, specifically, with the creation of text through elaborate systems.

The use of newer tools like Gemini, ChatGPT, Microsoft Copilot, and QuillBot has eased the process of writing and paraphrasing written work. Although these technologies offer great convenience, over-dependence on them presents a lot of problem. More and more students present academic papers, which are fully produced by AI, and business texts are written by professionals with minimum human intervention. This tendency poses a risk to the originality and authenticity of content and brings serious doubts concerning the misuse of it, such as the creation of fake documents.

As a result, the ability to detect the text produced by AI has been more critical than ever, particularly in situations where originality and trust are key components. The necessity to distinguish between the content created by humans and AI-generated content dictates the preference of this research, as it will help to address the risks of increased reliance on AI tools.

On various publicly available datasets, we have conducted our inquiry. We used an ensemble learning structure in which

we combined different classifiers and obtained a final prediction. The ensemble approach is, in a sense, the best method to identify AI-generated texts because human-written texts are diverse in terms of authorial style, whereas the AI-generated texts can take numerous forms in the context of various tools and platforms, each of which may take different stylistic features. A group approach is, thus, suitable to support such differences and provide strong forecasts. Our hybrid stacking classifier combines seven powerful base learners including XGBoost, LightGBM, Random Forest, Gradient Boosting, Extra Trees, Support Vector Machine and K-Nearest Neighbors, and it uses a logistic regression meta model to mix the results of these base learners.

Based on the textual information contained in the datasets we have designed relevant features to help in the task of classifications. We have selected the most appropriate features to be used in the analysis through the selection of features.

The further sections address the connected previous researches and our methodology and the corresponding results of the evaluation.

II. LITERATURE REVIEW

We have studied the existing researches on AI generated text detection to gain insights which can be very helpful.

Abburi et al. [1] have done AI generated text detection using an ensemble method. There were several datasets that were evaluated. Under the employed method, every input was fed through five pre-trained models which were DeBERTa large, RoBERTa with XNLI, RoBERTa large, RoBERTa base OpenAI detector, XLM-RoBERTa NLI. To maximize the benefits of the methods, separately generated class probabilities of each model were combined and given as an input to train a voting classifier of which were Logistic Regression, Random Forest, Gaussian Naive Bayes, Support Vector Machine. A maximum accuracy of the experiment was 78.4%.

Shah et al. [2] employed a combining method that has stylistic characteristics in the detection of AGTs. Various facets of Lexical features, Readability scores, Diversity and richness of the vocabulary were put into consideration. The researchers made custom datasets of the research. The initial classification in the experiment was Logistic Regression, Decision Tree, Random Forest, Support Vector Machine, Gradient Boosting,

after which the final score of the two best classifiers, the results were combined with two methods of xAI analysis, namely SHAP and LIME. The hybrid solution presented strong results.

Yan et al. [3] prepared a bespoke dataset and were able to use RoBERTa and SVM to detect AGT. RoBERTa model was fine tuned and SVM model was trained on a few features created by the in house scoring engine. RoBERTa was able to reach 99.75 percent and SVM reached 96 percent.

Mitchel et al. used probability curvature to detect machine generated texts [4]. The researchers conjectured that text generated by LLM is of negative form. They confirmed that small rewrites of the texts created by models are less likely to be generated under the model in comparison with the original text and vice versa to the case of human written texts.

In their work on AGTs detection with pre-trained models, Yadagiri et al. concentrated on various linguistic features [5]. The scholars developed a dataset containing about forty thousand queries and ChatGPT and human answers. CNN-BiLSTM, RNN, GPT-2, RoBERTa were the models in use. The strongest of the models had an accuracy of 99.73 which was RoBERTa.

Zhang et al. [6] took a hybrid method of AI generated texts detection. In hybrid model, Bayesian Classifier was used together with TF-IDF, SGD classifier, LightGBM classifier, CatBoost classifier and BPE tokenisation were used together with various instances of Bayesian classifier. Subsequently, 12 instances of DeBERTa v3 large models were used in this ensemble pipeline. The experimental method was the one to achieve the highest ROC-AUC of 97.5percent.

Qazi et al. [7] created a new dataset and suggested a model of GPT-generated text detection in English. The authors developed GPT Reddit Dataset (GRiD) that has 6513 samples, 1368 of which were created using the GPT-3.5-turbo model and the rest were created by humans. The human written texts were scraped on Reddit through the PRAW Python and the posts that were written prior to the official launch of the ChatGPT web application. The scientists suggested a new approach according to which they used tensor decomposition as the tool of recognizing fundamental patterns of data GPTEN. BERT, SVM and Random Forest as well as the proposed model were utilized by the scientists. BERT was more effective than other three.

Gryka and colleagues [8] in their research presented a set of machine-learning classifiers to compare them in relation to detecting artificially generated email content. All were considered: the decision-tree, random-forest (using a hundred estimators), multilayer-perceptron, k-nearest neighbours (k=5), support-vector machine (with radial-basis-function kernel), AdaBoost (fifty decision trees with learning rate=1), Gaussian Naive Bayes and quadratic discriminant analysis. The corpus was composed of over ten thousand English and Polish messages. Here the random-forest algorithm had most optimal performance with the F1 score of 98.8 per cent on English subset and 92.1 per cent on Polish subset.

Zhou and Wang [9] carried out an experiment on AI-generated prose detection in various fields. They combined

multiple existing corpora to form a composite dataset to be experimented with. Their contribution in terms of methodology was an adapted version of RoBERTa, called RoBERTa-Ranker, in which a loss margin-ranking function was added, and a mean-pooling layer was added to the encoder output, which further enriched the sentence representation before classification.

Guo et al. [10] suggested a multi-level contrastive learning and multi-task objective end-to-end pipeline to detect AI-generated text. First, training samples were pre-encoded and salient features were derived; and similarities between the encoded representations and a collection of feature vectors were compared to every textual instance. The last predictive step used a k -nearest neighbour algorithm to give classifications.

Wu and colleagues [11] proposed a set of large-language-model (LLM) generated content detectors. Their sample was made up of human-written passages filtered down to existing corpora and publicly available sites compared to the output of GPT-3.5-turbo, PaLM-2-bison, Claude-instant, and Llama-2-70B. This balanced mix was tested using systematic experiments to determine the performance of various classifiers.

Guo and co-workers [12] later had a deterministic view of causal language models. They maintained that such models when provided with data written by humans store a lot of contextual token data at the expense of making predictions about the next token. Their analysis model used bidirectional cross-entropy of generated logits and the ground-truth forward token, and logits and the previous backward token.

Wang et al. [13] used the GBERT architecture on the problem of identifying AI-generated text. They had a proprietary data set of 670 human generated samples and 708 algorithmically generated samples. The neural network had an average training and test accuracy of 98.1 and 97.71 respectively.

Yu et al. [14] were interested in the creation of a benchmark to recognize the abstracts created by ChatGPT. This amounted to 15,395 human-created abstracts and 35,304 machine-created ones, obtained via IEEE Xplore based on defined keywords. The latter were divided into three categories, namely Generation (completely algorithmic abstracts), Polish (text modeled and refined by a human), and Mix (hybrid model-polished and human text). A variety of BERT variants were tested on this dataset; the highest AUCs were achieved on Polish partition (99.56), Generation (100) and Mix (87.83).

Pudasaini et. al. [15] compared ArguGPT, RADAR (Robust AI Text Detector through Adversarial Learning) and OpenAI Detector in the framework of AI-generated text detection. They were evaluated based on M4 and HC3 datasets.

According to the assumptions expressed by McGovern and co-authors [16], it is possible to perceive AI-generated textual material with relatively simple classifiers, as long as informative features are extracted. They used a GradientBoost classifier and three separate sets of features including word n-grams, character n-grams, and part-of-speech n-grams. In several data sets, their algorithm gave encouraging detection rates.

Varadarajan et al. [17] reasoned that the text produced by LLM has poor human-like authenticity and richness. They laid stress on the significance of human qualities like demographics, personality, empathy as well as the behaviour linguistic characteristics. They used a wide variety of datasets, implementing an unsupervised classification pipeline, which used spectral clustering using a radial-basis kernel.

Andric et al. [18] presented a graph-based classification system that is taxonomy aware and a combination of text representation and graph neural networks between the identification of AI-generated content. They explored different methods of initializing feature of nodes and formed a co-occurrence graph. Language models that have been pre-trained (BERT-Base-Uncased or RoBERTa-Base) were fine-tuned. The resulting graph embeddings were subsequently passed through a classifier which consisted of a dense neural network layer. The Autexification-2023 dataset was used as an experimental foundation.

Abbas [19] investigated the identification of AI-generated text using a zero-shot prompting approach with the help of Sentence-Bert embeddings. Also a graph convolutional network was incorporated, thus increased accuracy was achieved. All empirical assessments were based on the PAN-AP-2019 data.

In a similar work, the authors [20] created a dataset of 509 human and AI-generated questionanswer pairs and tested them using several machine based and deep based classification models. Their approach that uses RoBERTa demonstrated high performance as compared to the other architectures that were tested.

Alhijawi and colleagues [21] selected a collection of 3,000 records, which were in turn divided into human-written, AI-generated as well as mixed content: 1,000 records each. They tested a hybrid architecture of multilayer perceptron and convolutional neural network elements and finally achieved a top-ranking accuracy of 0.876 in the experiment.

III. METHODOLOGY

A. The Dataset

We have used two publicly available datasets from the Kaggle platform. The datasets have been experimented with separately.

- Dataset 01: *DAIGT V2 Dataset* [22] is the first dataset that we have done our research with. It has been uploaded by Darek Kłeczek [23] for AI Generated text detection. The dataset has 44868 text records and 27371 records are human written while the rest are AI generated. The AI generated texts are labeled as 1 while texts written by humans are labeled as 0. The dataset is available in CSV file format.
- Dataset 02: *LLM - Detect AI Generated Text Dataset* [24] is the second dataset that we have used. It was uploaded by Sunil Thite [25]. 0 is used to indicate

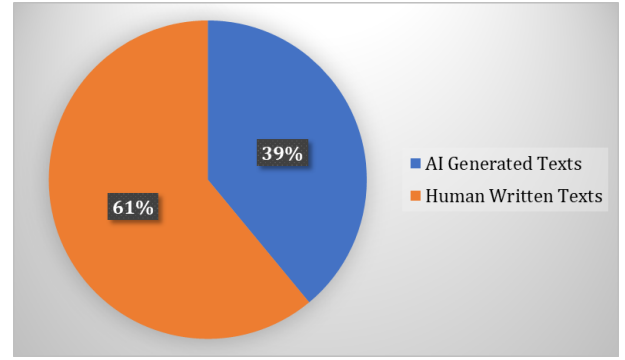


Fig. 1. Percentage of Category wise Records in the *DAIGT V2 Dataset*

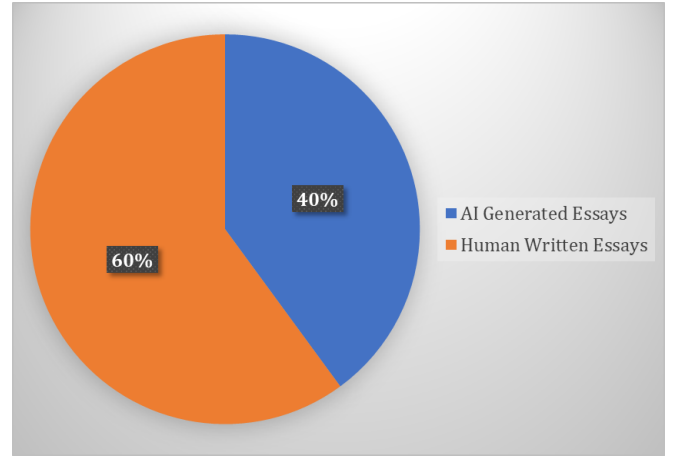


Fig. 2. Percentage of Category wise Records in the *LLM - Detect AI Generated Text Dataset*

the Human Written Essays whereas 1 is for the AI Generated Essays. 17508 essays are written by humans and 11637 essays are AI generated.

B. Application of the Hybrid Approach

1) *Pre-processing*: We have performed the standard pre-processing of a dataset before using it.

- Removal missing values: We have removed any row where either the text or label column is missing.
- Tokenization of Texts: Each text is tokenized into sentences and they are further tokenized into words.
- Lower-casing of the text: The tokens have been converted to lower case.
- Expansion of contractions: Appropriate contractions have been added for consistency.
- Recognition of Named Entity: Tokens have been tagged with their part-of-speech to detect named entities.

2) *Feature Engineering*: We have created a feature matrix of different Hand-crafted statistical & linguistic features and Dense semantic features from BERT embeddings. The features are mentioned below.

- Basic length & frequency features
 - Total number of tokens in the text
 - Number of distinct tokens
 - Total number of characters in text
 - Average number of characters per token
 - Ratio of unique words to total words
 - Fraction of words that occur only once
 - Number of sentences in the text
 - Average number of words per sentence
- Punctuation & stopword features
 - Number of punctuation characters
 - Number of tokens that are English stopwords
- Contraction-related feature: Number of added contractions
- Emotion-related features: Number of words matching any particular emotion
- Sentiment features
 - Sentiment polarity score
 - VADER compound sentiment score
 - Subjectivity score
- Readability features
 - Flesch Reading Ease score
 - Gunning fog index
- Pronoun features
 - Number of first person pronouns
 - Number of second person pronouns
- Named Entity Recognition (NER) features
 - Number of named entities tagged as PERSON
 - Number of named entities tagged as DATE
- N-gram uniqueness features
 - Ratio of unique bigrams to total bigrams
 - Ratio of unique trigrams to total trigrams
- Syntax variety feature: Fraction of unique part-of-speech tags relative to total words
- Perplexity feature: Perplexity score for identifying predictability patterns
- BERT Embedding feature: Each text is converted into a 768-dimensional vector using the pretrained bert-base-uncased model. The text is tokenized, truncated to 512 tokens, passed through BERT, and the [CLS] token's embedding is extracted to represent the entire sequence.

For each handcrafted feature, our implementation calculates the Pearson correlation coefficient between that feature's values and the label vector. Only features with $|correlation|$ greater than 0.1 are kept. BERT embeddings are not filtered this way, rather they're added in full later.

3) *Splitting and Standardization of the Dataset*: The dataset is split into train, validation, and test sets with the ratio of 70% for training, 10% for validation, and 20% for testing purposes. After splitting, a Standard Scaler is fitted on the training features to compute the mean and standard deviation for each feature, and these statistics are then

applied to transform the training, validation, and test sets. This standardization ensures that all features have zero mean and unit variance, which helps to train more effectively and converge faster.

4) *The Stacking Ensemble Classifier*: In our implementation, the classification model is implemented as a stacking ensemble, which combines multiple different base models and then uses a meta model to make the final prediction.

- Base models (Level-0 learners)
 - The XGBoost classifier is tuned for a balance between high capacity and regularization. It uses `n_estimators = 300` to allow sufficient boosting rounds, paired with a deep tree depth of `max_depth = 10` to capture complex feature interactions. A low `learning_rate = 0.03` ensures gradual learning, reducing the risk of overfitting, while `subsample = 0.9` and `colsample_bytree = 0.8` add randomness for better generalization. Regularization is handled by `reg_alpha = 0.5` (L1 penalty for sparsity) and `reg_lambda = 1` (L2 penalty for stability). The evaluation metric is set to `mlogloss`, and the label encoder is disabled for modern usage.
 - The LightGBM classifier mirrors XGBoost in spirit but is optimized for speed and scalability. It also uses `n_estimators = 300` with a slightly smaller `max_depth = 9` and a controlled leaf growth using `num_leaves = 100`. The `learning_rate = 0.03` allows steady convergence, while `subsample = 0.9` and `colsample_bytree = 0.8` limit overfitting through stochasticity. Lighter regularization is applied with `reg_alpha = 0.1` and `reg_lambda = 0.1`, making it more flexible than XGBoost in splitting patterns.
 - The Random Forest classifier is tuned for robust bagging with `n_estimators = 300` and `max_depth = 20` to allow deep, information-rich trees. A `min_samples_leaf = 3` prevents overly specific leaves, ensuring smoother decision boundaries. The inclusion of `class_weight = 'balanced'` automatically adjusts for class imbalance by weighting minority classes more heavily.
 - The Gradient Boosting classifier is configured to be more conservative, with `n_estimators = 250` and a `learning_rate = 0.05`, allowing it to incrementally refine predictions over fewer boosting stages. The `max_depth = 5` keeps trees shallow to prevent overfitting, and `min_samples_split = 5` ensures that splits only occur when enough samples are available, further regularizing the model.
 - The Extra Trees classifier introduces high randomness into the ensemble by setting `max_features = 'sqrt'`, meaning each split only considers the square root of available features. With `n_estimators = 300`, it builds many randomized trees, and `class_weight = 'balanced'` keeps it fair across imbalanced labels.

- The Support Vector Classifier (SVC) is configured with $C = 5$, giving it a strong focus on correctly classifying training samples by reducing the penalty for margin violations. The kernel = ‘rbf’ captures complex non-linear relationships in the feature space, and probability = True enables probability outputs required for stacking. Class balancing is again enforced through class_weight = ‘balanced’.
 - The K-Nearest Neighbors classifier is set with n_neighbors = 5 to use a modest neighborhood size, and weights = ‘distance’ ensures that nearer points have a stronger influence on classification decisions, making it adaptive to local patterns.
- Meta model (Level-1 learner): The meta-model is Logistic Regression, a simple but effective linear classifier. It is the final layer of decision-making. It applies l2 (ridge) regularization to prevent overfitting while maintaining all features. It also employs balanced class weight to ensure fair treatment of all classes when learning how to optimally blend base model predictions. The model takes the following as inputs:
 - The predictions of each base model
 - The original features

Using both the base models’ outputs and the original data improve accuracy because the meta-model can learn when to trust each base model’s prediction.

- Training process of the Stacking mechanism: The training data is split into 5 folds. Each base model is trained on 4 folds and makes predictions on the remaining fold. These out-of-fold predictions become new features for the meta-model. After this, the base models are retrained on the entire training set. The meta-model is trained on the out-of-fold predictions along with the original features. For the test set, base models predict first, and those predictions along with the original features are passed to the meta-model for final classification.

C. Evaluation Results

For evaluation of the text classification task, we have used Accuracy, Precision, Recall and F1 score metrics. The performance outcomes of our ensemble model on the both datasets are illustrated in the Table 1.

TABLE I
EVALUATED RESULTS OF THE PROPOSED METHOD

Dataset	Accuracy	Precision	Recall	F1 Score
Dataset 01	0.988	0.987	0.988	0.987
Dataset 02	0.994	0.994	0.994	0.994

We can observe that the model has been able to achieve excellent results over the both datasets.

Our methodology achieves great results as it combines strong feature engineering, rich embeddings, thoughtful feature selection, and a highly diverse, well-tuned stacking ensemble — each of which reinforces the others. The implementation approach does not rely on text alone. The pipeline combines BERT embeddings with handcrafted linguistic features. This hybrid representation captures both deep semantic meaning and surface-level stylistic patterns, giving the models multiple complementary “views” of the same text. The correlation between each handcrafted feature and the target labels are calculated, keeping only those with meaningful relationships. This reduces noise, removes irrelevant features, and ensures that the machine learning models focus on the most predictive aspects. Combined with the high information density of BERT embeddings, the input space is rich but not unnecessarily large.

Standardizing features ensures that models sensitive to scale perform optimally. Stratified splitting preserves the class distribution in training, validation, and test sets, avoiding skewed evaluation results. Seven very different algorithms are combined into a meta-learning framework. The meta-model receives both the predictions from base models and the original features, preventing information loss and allowing it to correct base model errors. The diversity of the base learners means that their errors are less likely to overlap, so the meta-model can learn an optimal blend.

Each base learner is tuned for a balance between capacity and generalization. Boosting models are set with low learning rates for stability, while bagging models are set to build many deep trees for robustness. SVM and k-NN settings are chosen to handle complex patterns without becoming unstable. This careful tuning ensures no model is overfitting badly, so stacking benefits from genuinely complementary predictions. Multiple models are trained with balanced class weighting, meaning minority classes aren’t ignored. This is particularly important in text classification tasks where one class may dominate.

The individual models are strong, but their prediction patterns differ, which is exactly what stacking needs. The logistic regression meta-model learns to trust certain models in certain scenarios, and having access to the original features gives it an extra safety net.

IV. CONCLUSION AND FUTURE WORKS

Identification of AI generated text is a crucial task in today’s life. Due to heavy usage of internet and AI tools, it is quite difficult to identify such texts. Rather than using standalone classifiers, we have used ensemble learning technique for the classification task and it has yielded excellent result with accuracy. In future, we hope to perform the task on specialized texts. Also, we hope to explore possibilities and scopes for improving the evaluation outcome of the employed model.

REFERENCES

- [1] H. Abburi, K. Roy, M. Suesserman, N. Pudota, B. Veeramani, E. Bowen, and S. Bhattacharya, "A simple yet efficient ensemble approach for AI-generated text detection," in *Proc. 2023 Conf. Empirical Methods in Natural Language Processing*, 2023, p. 413.
- [2] A. Shah, P. Ranka, U. Dedhia, S. Prasad, S. Muni, and K. Bhowmick, "Detecting and unmasking AI-generated texts through explainable artificial intelligence using stylistic features," *Int. J. Adv. Comput. Sci. Appl.*, vol. 14, no. 10, 2023.
- [3] D. Yan, M. Fauss, J. Hao, and W. Cui, "Detection of AI-generated essays in writing assessments," *Psychol. Test Assess. Model.*, vol. 65, no. 1, pp. 125–144, 2023.
- [4] E. Mitchell, Y. Lee, A. Khazatsky, C. D. Manning, and C. Finn, "DetectGPT: zero-shot machine-generated text detection using probability curvature," in *Proc. 40th Int. Conf. Mach. Learn.*, 2023, pp. 24950–24962.
- [5] A. Yadagiri, L. Shree, S. Parween, A. Raj, S. Maurya, and P. Pakray, "Detecting AI-generated text with pre-trained models using linguistic features," in *Proc. 21st Int. Conf. Natural Language Processing (ICON)*, 2024, pp. 188–196.
- [6] Y. Zhang, Q. Leng, M. Zhu, R. Ding, Y. Wu, J. Song, and Y. Gong, "Enhancing text authenticity: A novel hybrid approach for AI-generated text detection," in *Proc. 2024 IEEE 4th Int. Conf. Electron. Technol., Commun. Inf. (ICETCI)*, 2024, pp. 433–438.
- [7] Z. Qazi, W. Shiao, and E. E. Papalexakis, "GPT-generated text detection: Benchmark dataset and tensor-based detection method," in *Companion Proc. ACM Web Conf. 2024*, 2024, pp. 842–846.
- [8] P. Gryka, K. Gradoń, M. Kozłowski, M. Kutyla, and A. Janicki, "Detection of AI-generated emails—A case study," in *Proc. 19th Int. Conf. Availability, Reliability and Security*, 2024, pp. 1–8.
- [9] Y. Zhou and J. Wang, "Detecting AI-generated texts in cross-domains," in *Proc. ACM Symp. Document Eng. 2024*, 2024, pp. 1–4.
- [10] X. Guo, Y. He, S. Zhang, T. Zhang, W. Feng, H. Huang, and C. Ma, "Detective: Detecting AI-generated text via multi-level contrastive learning," *Adv. Neural Inf. Process. Syst.*, vol. 37, pp. 88320–88347, 2024.
- [11] J. Wu, R. Zhan, D. Wong, S. Yang, X. Yang, Y. Yuan, and L. Chao, "DetectRL: Benchmarking LLM-generated text detection in real-world scenarios," *Adv. Neural Inf. Process. Syst.*, vol. 37, pp. 100369–100401, 2024.
- [12] H. Guo, S. Cheng, X. Jin, Z. Zhang, K. Zhang, G. Tao, G. Shen, and X. Zhang, "BiScope: AI-generated text detection by checking memorization of preceding tokens," *Adv. Neural Inf. Process. Syst.*, vol. 37, pp. 104065–104090, 2024.
- [13] H. Wang, J. Li, and Z. Li, "AI-generated text detection and classification based on BERT deep learning algorithm," *Theor. Nat. Sci.*, vol. 39, pp. 311–316, 2024.
- [14] P. Yu, J. Chen, X. Feng, and Z. Xia, "CHEAT: A large-scale dataset for detecting ChatGPT-written abstracts," *IEEE Trans. Big Data*, 2025.
- [15] S. Pudasaini, L. Miralles, D. Lillis, and M. L. Salvador, "Benchmarking AI text detection: Assessing detectors against new datasets, evasion tactics, and enhanced LLMs," in *Proc. 1st Workshop GenAI Content Detection (GenAIDetect)*, 2025, pp. 68–77.
- [16] H. McGovern, R. Stureborg, Y. Suhara, and D. Alikaniotis, "Your large language models are leaving fingerprints," in *GenAIDetect 2025*, 2025, p. 85.
- [17] V. Varadarajan, S. Giorgi, S. Mangalik, N. Soni, D. M. Markowitz, and H. A. Schwartz, "The consistent lack of variance of psychological factors expressed by LLMs and spambots," in *Proc. 1st Workshop GenAI Content Detection (GenAIDetect)*, 2025, pp. 111–119.
- [18] A. Valdez-Valenzuela, H. Gómez-Adorno, and M. Montes-y-Gómez, "Text graph neural networks for detecting AI-generated content," in *Proc. 1st Workshop on GenAI Content Detection (GenAIDetect)*, 2025, pp. 134–139.
- [19] H. M. Abbas, "A novel approach to automated detection of AI-generated text," *J. Al-Qadisiyah Comput. Sci. Math.*, vol. 17, no. 1, pp. 1–17, 2025.
- [20] M. M. Oghaz, L. B. Saheer, K. Dhame, and G. Singaram, "Detection and classification of ChatGPT-generated content using deep transformer models," *Front. Artif. Intell.*, vol. 8, p. 1458707, 2025.
- [21] B. Alhijawi, R. Jarrar, A. AbuAlRub, and A. Bader, "Deep learning detection method for large language models-generated scientific content," *Neural Comput. Appl.*, vol. 37, no. 1, pp. 91–104, 2025.
- [22] "DAIGT V2 Train Dataset", [Online] Available: <https://www.kaggle.com/datasets/thedrcat/daigt-v2-train-dataset/data>, Last Accessed on: 09 August 2025.
- [23] "Darek Kłeczek", [Online] Available: <https://www.kaggle.com/thedrcat>, Last Accessed on: 09 August 2025.
- [24] "LLM - Detect AI Generated Text Dataset", [Online] Available: <https://www.kaggle.com/datasets/sunilthite/llm-detect-ai-generated-text-dataset>, Last Accessed on: 09 August 2025.
- [25] "Sunil Thite", [Online] Available: <https://www.kaggle.com/sunilthite>, Last Accessed on: 09 August 2025.